R: limpieza, manipulación y exploración de datos

Cheatsheet de `stringr`

Jonatán Perren

26 de marzo de 2026

El paquete `stringr`

Una interfaz consistente para cadenas de texto

El paquete `stringr` forma parte del ecosistema tidyverse y proporciona un conjunto de funciones coherentes para manipular cadenas de texto en R.

Todas las funciones comienzan con el prefijo `str_`, lo que facilita su descubrimiento y memorización.

¿Por qué usar `stringr` en lugar de R Base?

- Nombres consistentes y predecibles.
- El primer argumento siempre es la cadena (compatible con el pipe ⊳).
- Manejo uniforme de `NA` (devuelve `NA`, no errores).
- Construido sobre la librería `ICU`, más robusta para Unicode.

```r
# install.packages("stringr")
library(stringr)
```

Longitud y subcadenas

str_length: Contar caracteres

A diferencia de `nchar()`, maneja `NA` de forma segura.

```
str_length(c("Hola", "Economía", NA))
```

```
[1]  4  8 NA
```

str_sub: Extraer o reemplazar subcadenas

```
x <- "Universidad"
str_sub(x, 1, 4)
```

```
[1] "Univ"
```

Los índices negativos cuentan desde el final.

```
str_sub(x, -3, -1) # Últimos 3 caracteres
```

```
[1] "dad"
```

```
str_sub(x, 1, 4) <- "UNIV" # Reemplazo in-place
x
```

```
[1] "UNIVersidad"
```

Conversión de mayúsculas y minúsculas

stringr ofrece cuatro funciones de conversión de caso, más robustas que **toupper()/tolower()** de R Base para caracteres Unicode.

```r
x <- "facultad de ciencias económicas, universidad nacional del litoral"

str_to_upper(x)
```

```
[1] "FACULTAD DE CIENCIAS ECONÓMICAS, UNIVERSIDAD NACIONAL DEL LITORAL"
```

```r
str_to_lower("LICENCIATURA EN ECONOMÍA")
```

```
[1] "licenciatura en economía"
```

```r
str_to_title(x) # Primera letra de cada palabra
```

```
[1] "Facultad De Ciencias Económicas, Universidad Nacional Del Litoral"
```

```r
str_to_sentence("LiCenCIAtura eN EcOnomíA") # Solo la primera letra
```

```
[1] "Licenciatura en economía"
```

Espacios en blanco: str_trim, str_squish y str_pad

str_trim: Eliminar espacios laterales

```r
str_trim("   texto con espacios             ")
```

```
[1] "texto con espacios"
```

```r
str_trim("   texto   ", side = "left") # Solo izq.
```

```
[1] "texto   "
```

str_squish: Limpiar todos los espacios

Elimina laterales **y** colapsa múltiples espacios internos en uno solo. Ideal para procesar las observaciones cargadas manualmente.

```r
str_squish("  datos    muy   sucios  ")
```

```
[1] "datos muy sucios"
```

str_pad: Rellenar hasta un ancho

```r
str_pad(c("1", "23", "456"), width = 5, pad = "0")
```

```
[1] "00001" "00023" "00456"
```

Truncar y ajustar: str_trunc y str_wrap

str_trunc: Cortar cadenas largas

Trunca al ancho indicado, añadiendo puntos suspensivos.

```r
x <- "Facultad de Ciencias Económicas"

str_trunc(x, width = 15)
```

```
[1] "Facultad de ..."
```

```r
str_trunc(x, width = 15, side = "left")
```

```
[1] "...Económicas"
```

str_wrap: Ajustar texto a un ancho de línea

Inserta saltos de línea para ajustar un párrafo a un ancho determinado.

```r
cat(str_wrap(
  "Este es un texto largo para un gráfico",
  width = 20
))
```

```
Este es un texto
largo para un
gráfico
```

Concatenar y unir cadenas

str_c: Concatenar cadenas (como `paste0`)

```r
str_c("Precio", "USD", sep = ": ")
```

```
[1] "Precio: USD"
```

```r
str_c("Ítem", 1:3, sep = "-")
```

```
[1] "Ítem-1" "Ítem-2" "Ítem-3"
```

str_dup: Duplicar una cadena

```r
str_dup("ab", 3)
```

```
[1] "ababab"
```

str_flatten: Colapsar un vector en una sola cadena

```
provincias ← c("Santa Fe", "Córdoba", "Buenos Aires")
str_flatten(provincias, collapse = ", ")
```

```
[1] "Santa Fe, Córdoba, Buenos Aires"
```

```
str_flatten(provincias, collapse = ", ", last = " y ")
```

```
[1] "Santa Fe, Córdoba y Buenos Aires"
```

Interpolación con str_glue

`str_glue()` permite insertar expresiones de R directamente dentro de una cadena usando llaves `{ ... }`.

```r
nombre <- "Jonatán"
nota   <- 8.5

str_glue("El alumno {nombre} obtuvo {nota}")
```

```
El alumno Jonatán obtuvo 8.5
```

```r
#Aunque sea un despropósito, también se podría hacer esto.
str_glue("Nota: {round(nota, 0)} ({if(nota ≥ 6) 'Aprobado' else 'Desaprobado'})")
```

```
Nota: 8 (Aprobado)
```

str_glue_data: Interpolar desde un data frame

```r
sectores <- data.frame(
  sector = c("Agro", "Industria"),
  ventas = c(210.3, 120.5)
)

str_glue_data(sectores, "{sector}: ${ventas}M")
```

```
Agro: $210.3M
Industria: $120.5M
```

Detectar patrones: str_detect

`str_detect()` devuelve un vector lógico. Es la versión `stringr` de `grepl()`.

Es especialmente útil dentro de `filter()` de `dplyr`.

```
sectores <- c("Industria", "Servicios", "Agro",
              "Comercio", "Industria pesada")

str_detect(sectores, "Industria")
```

```
[1]  TRUE FALSE FALSE FALSE  TRUE
```

```
# Negar la detección
str_detect(sectores, "Industria", negate = TRUE)
```

```
[1] FALSE  TRUE  TRUE  TRUE FALSE
```

```
# Uso típico con dplyr
datos ▷ filter(str_detect(sector, "Ind"))
```

Detectar patrones: str_starts, str_ends y str_which

str_starts y str_ends

Versiones especializadas que verifican inicio o final.

```
x <- c("inflación", "ingreso", "exportación",
       "importación")

str_starts(x, "in")
```

```
[1]  TRUE  TRUE FALSE FALSE
```

```
str_ends(x, "ción")
```

```
[1]  TRUE FALSE  TRUE  TRUE
```

str_which: Posiciones de las coincidencias

Devuelve los índices (no lógicos), como `grep()`.

```
str_which(x, "ción")
```

```
[1] 1 3 4
```

Contar coincidencias: str_count

`str_count()` cuenta cuántas veces aparece un patrón en cada elemento.

```
texto <- c("banana", "manzana", "ananá")
str_count(texto, "an")
```

```
[1] 2 2 2
```

```
# Contar vocales
str_count(c("Economía", "R"), "[aeiouáéíóú]")
```

```
[1] 5 0
```

Útil para generar variables en un data frame:

```
datos |>
  mutate(n_palabras =
    str_count(descripcion, "\\W+") + 1)
```

Estas últimas dos líneas usan algo llamado «expresiones regulares». En general, está bien saber que existen para, en ocasiones, orientar solicitudes en ese sentido. Pero aprender a manejarlas al 100 %, si no se usan con frecuencia, es un despropósito. Sí conviene aprender a buscar patrones regulares. Las siguientes funciones hacen uso de ellas.

Extraer coincidencias: str_extract y str_extract_all

str_extract: Primera coincidencia

```r
x <- c("Precio: $150", "Costo: $80", "Sin dato", "135")

str_extract(x, "\\$[0-9]+")
```

```
[1] "$150" "$80"  NA    NA
```

```r
# Extraer solo los números
str_extract(x, "[0-9]+")
```

```
[1] "150" "80"  NA    "135"
```

str_extract_all: Todas las coincidencias

```r
str_extract_all("Tel: 342-155-1234 / 351-123",
                "[0-9]+")
```

```
[[1]]
[1] "342" "155" "1234" "351" "123"
```

Subconjunto: str_subset

`str_subset()` devuelve los **elementos** que coinciden (no lógicos ni posiciones).

```r
ciudades <- c("Santa Fe", "San Luis", "Córdoba",
              "San Juan", "Rosario")

str_subset(ciudades, "San")
```

```
[1] "Santa Fe" "San Luis" "San Juan"
```

```r
# Con negate
str_subset(ciudades, "San", negate = TRUE)
```

```
[1] "Córdoba" "Rosario"
```

Para filtrar filas de un data frame usá `filter(str_detect( ... ))` en lugar de `str_subset()`, que opera sobre vectores.

Localizar coincidencias: str_locate

Devuelve una matriz con las posiciones de inicio y fin. Útil para un procesamiento fino o junto con `str_sub()`.

str_locate: Primera coincidencia

```
str_locate("Facultad de Ciencias", "de")
```

```
     start end
[1,]    10  11
```

str_locate_all: Todas las coincidencias

```
str_locate_all("banana", "an")
```

```
[[1]]
     start end
[1,]     2   3
[2,]     4   5
```

Reemplazar: str_replace y str_replace_all

str_replace: Primera coincidencia

```r
str_replace("2025-01-15", "-", "/")
```

```
[1] "2025/01-15"
```

str_replace_all: Todas las coincidencias

```r
str_replace_all("2025-01-15", "-", "/")
```

```
[1] "2025/01/15"
```

```r
# Reemplazos múltiples con vector con nombre
str_replace_all(
  "Ingresos: 1000 USD",
  c("USD" = "ARS", "1000" = "1200000")
)
```

```
[1] "Ingresos: 1200000 ARS"
```

str_replace_na: Reemplazar NA

```r
str_replace_na(c("dato", NA, "otro"), "Sin dato")
```

```
[1] "dato"     "Sin dato" "otro"
```

Eliminar patrones: str_remove y str_remove_all

Son atajos para `str_replace(x, patron, "")`.

str_remove: Primera coincidencia

```
str_remove("$1.250,00", "\\$")
```

```
[1] "1.250,00"
```

str_remove_all: Todas las coincidencias

Ideal para limpiar datos importados.

```
precios <- c("$1.250,00", "$800,50", "$3.100,00")

precios |>
  str_remove_all("\\$") |>
  str_remove_all("\\.") |>
  str_replace_all(",", ".") |>
  as.numeric() #Ojo! Es solo un ejemplo. Hay formas mejores.
```

```
[1] 1250.00  800.50 3100.00
```

Dividir cadenas: str_split

str_split: Devuelve una lista

```r
str_split("Santa Fe-Córdoba-Rosario", "-")
```

```
[[1]]
[1] "Santa Fe" "Córdoba" "Rosario"
```

str_split_fixed: Devuelve una matriz

Útil cuando se necesita un número fijo de columnas.

```r
str_split_fixed(
  c("Perren, Jonatán", "García, Ana"),
  ", ",
  n = 2
)
```

```
     [,1]     [,2]
[1,] "Perren" "Jonatán"
[2,] "García" "Ana"
```

str_split_i: Extraer la i-ésima parte

```r
str_split_i("2025-01-15", "-", i = 2) # El mes
```

```
[1] "01"
```

Grupos de captura: str_match

A diferencia de `str_extract()`, `str_match()` captura subgrupos definidos con paréntesis en la regex.

str_match: Primera coincidencia

```r
x <- c("Juan (25 años)", "Ana (30 años)")

str_match(x, "(\\w+) \\((\\d+) años\\)")
```

```
     [,1]             [,2]   [,3]
[1,] "Juan (25 años)" "Juan" "25"
[2,] "Ana (30 años)"  "Ana"  "30"
```

Cada columna extra corresponde a un grupo de captura.

str_match_all: Todas las coincidencias

```r
str_match_all(
  "datos-2024 y datos-2025",
  "(\\w+)-(\\d{4})"
)
```

```
[[1]]
     [,1]          [,2]    [,3]
[1,] "datos-2024" "datos" "2024"
[2,] "datos-2025" "datos" "2025"
```

Ordenar cadenas: str_sort y str_order

str_sort: Ordenar alfabéticamente

```r
x <- c("Córdoba", "Buenos Aires", "Santa Fe",
       "Álvarez")

str_sort(x)
```

```
[1] "Álvarez"      "Buenos Aires" "Córdoba"
[4] "Santa Fe"
```

```r
str_sort(x, decreasing = TRUE)
```

```
[1] "Santa Fe"     "Córdoba"      "Buenos Aires"
[4] "Álvarez"
```

str_order: Índices del orden

Análogo a `order()` pero con soporte Unicode.

```r
str_order(x)
```

```
[1] 4 2 1 3
```

str_unique: Valores únicos

```r
str_unique(c("a", "b", "a", "c", "b"))
```

Comparar e igualar: str_equal

`str_equal()` compara cadenas de forma consciente de la localización (locale).

```
str_equal("café", "café")
```

```
[1] TRUE
```

```
# Comparación ignorando mayúsculas/minúsculas
str_equal("Economía", "economía",
           ignore_case = TRUE)
```

```
[1] TRUE
```

Patrones y expresiones regulares

Muchas funciones de `stringr` aceptan **expresiones regulares** (regex) como patrón de búsqueda. Por ejemplo:

```r
# Extraer números de un texto
str_extract("Precio: $150", "[0-9]+")
```

```
[1] "150"
```

También se pueden usar **modificadores** para cambiar cómo se interpreta el patrón:

```r
# Ignorar mayúsculas/minúsculas
str_detect("INFLACIÓN",
  regex("inflación", ignore_case = TRUE))
```

```
[1] TRUE
```

```r
# Coincidencia literal (sin interpretar regex)
str_detect("3.14", fixed("."))
```

```
[1] TRUE
```

Para una guía completa de expresiones regulares, modificadores (`regex()`, `fixed()`, `coll()`, `boundary()`) y `str_view()`, consultá **mis slides con un resumen de todo lo que se puede hacer y cómo**, o también tenés el **cheatsheet de expresiones regulares**.

Ejemplo integrador (I): Limpieza de datos

Datos con provincias mal cargadas:

```r
provincias <- c(
  "  BUENOS      AIRES ",
  "cOrdoba",
  "santa fe  ",
  "entre ríos",
  " TUCUMAN  "
)

provincias |>
  str_squish() |>
  str_to_title()
```

```
[1] "Buenos Aires" "Cordoba"      "Santa Fe"
[4] "Entre Ríos"   "Tucuman"
```

Ejemplo integrador (II): Con dplyr

```r
library(dplyr)

empresas <- tibble(
  id = 1:5,
  nombre = c("ACME S.A.", "globant s.r.l.", "MERCADO libre", "YPF S.A.", "arcor"),
  ventas_txt = c("$1.200,50", "$800,00", "$15.300,75", "$25.000,00", "$3.450,25")
)

empresas %>
  mutate(
    nombre_limpio = nombre %>
      str_squish() %>
      str_to_title(),
    tipo = str_extract(nombre,
      regex("s\\.a\\.|s\\.r\\.l\\.",
            ignore_case = TRUE)) %>
      str_to_upper(),
    ventas = ventas_txt %>
      str_remove_all("\\$|\\.") %>
      str_replace(",", ".") %>
      as.numeric()
  )
```

Ejemplo integrador (III): Resultado

```
# A tibble: 5 × 6
id nombre           ventas_txt nombre_limpio  tipo   ventas
  <int> <chr>            <chr>      <chr>          <chr>   <dbl>
1     1 "  ACME S.A."    $1.200,50  Acme S.a.      S.A.    1200.
2     2 "globant s.r.l." $800,00    Globant S.r.l. S.R.L.   800
3     3 "MERCADO libre"  $15.300,75 Mercado Libre  NA     15301.
4     4 "YPF S.A."       $25.000,00 Ypf S.a.       S.A.   25000
5     5 "arcor"          $3.450,25  Arcor          NA      3450.
```

Resumen de funciones

Longitud y subcadenas

`str_length()`, `str_sub()`, `str_dup()`.

Caso y espacios

`str_to_upper()`, `str_to_lower()`, `str_to_title()`, `str_to_sentence()`, `str_trim()`, `str_squish()`, `str_pad()`, `str_trunc()`, `str_wrap()`.

Concatenar y unir

`str_c()`, `str_flatten()`, `str_glue()`, `str_glue_data()`.

Detectar y contar

`str_detect()`, `str_starts()`, `str_ends()`, `str_which()`, `str_count()`.

Extraer y localizar

`str_extract()`, `str_extract_all()`, `str_subset()`, `str_locate()`, `str_match()`, `str_match_all()`.

Reemplazar y eliminar

`str_replace()`, `str_replace_all()`, `str_remove()`, `str_remove_all()`, `str_replace_na()`.

Dividir, ordenar y comparar

`str_split()`, `str_split_fixed()`, `str_split_i()`, `str_sort()`, `str_order()`, `str_unique()`, `str_equal()`.

Equivalencias con R Base

Operación	stringr	R Base
Detectar	`str_detect()`	`grepl()`
Posiciones	`str_which()`	`grep()`
Extraer	`str_extract()`	`regmatches()`
Reemplazar 1	`str_replace()`	`sub()`
Reemplazar +	`str_replace_all()`	`gsub()`
Dividir	`str_split()`	`strsplit()`
Longitud	`str_length()`	`nchar()`
Subcadena	`str_sub()`	`substr()`
Concatenar	`str_c()`	`paste0()`
Mayúsculas	`str_to_upper()`	`toupper()`
Minúsculas	`str_to_lower()`	`tolower()`
Trim	`str_trim()`	`trimws()`